

Extreme Learning Machine with Kernels for Solving Elliptic Partial Differential Equations

Summary of Li, Liu, and Xiao (2023)

1 Introduction

Solving partial differential equations (PDEs) using machine learning methods has attracted increasing attention due to the strong nonlinear approximation abilities of these methods. While various machine learning approaches have been employed for solving PDEs, including artificial neural networks and wavelet neural networks, the Extreme Learning Machine (ELM) represents a unique approach due to its efficiency. This paper introduces a new method for solving elliptic partial differential equations based on an extreme learning machine with kernels (KELM), which offers significant advantages in terms of computational efficiency and accuracy.

Traditional neural network methods for solving PDEs, including those pioneered by Lagaris et al., face challenges related to the gradient-based optimization processes used to train their parameters. Specifically, these methods may suffer from convergence to local optima, require extensive computational resources, and involve tuning numerous parameters. The KELM approach overcomes these limitations through a unique learning mechanism that avoids iterative optimization.

2 Mathematical Foundations of KELMs

The Extreme Learning Machine (ELM) is a single hidden layer neural network where the input layer weights and hidden layer biases are randomly assigned, and only the output weights need to be determined. For a multi-input, single-output problem with input samples $X = (X_j)_{j=1,2,\dots,N}$ and output samples $T = (t_j)_{j=1,2,\dots,N}$, an ELM can be mathematically expressed as:

$$o_j = \sum_{i=1}^L \beta_i g(W_i \cdot X_j + \gamma_i) \approx t_j, j = 1, \dots, N \quad (1)$$

where o_j is the output variable, β_i is the weight of the output layers, $g(x)$ is an activation function (typically sigmoid: $g(x) = \frac{1}{1+e^{-x}}$), W_i is the weight

between the input layers and the i -th hidden neuron, γ_i is the threshold of the i -th hidden neuron, and L represents the number of neurons in the hidden layers.

The ELM learning process aims to minimize the output error:

$$\sum_{j=1}^N \|o_j - t_j\| = 0 \quad (2)$$

This equation can be rewritten in matrix form as:

$$H\beta = T \quad (3)$$

where H is the hidden layer output matrix. Once the input layer weight W and hidden layer threshold γ are randomly given, the output weight β can be analytically determined as:

$$\beta = H^\dagger T \quad (4)$$

where $H^\dagger$ is the Moore-Penrose generalized inverse of matrix H .

To improve stability and generalization, a regularization coefficient λ is introduced:

$$\beta = H^T(HH^T + \lambda I)^{-1}T \quad (5)$$

The kernel-based ELM (KELM) further extends this approach by introducing a kernel function to replace the activation function:

$$o_j = \begin{bmatrix} K(X_j, X_1) \\ K(X_j, X_2) \\ \vdots \\ K(X_j, X_N) \end{bmatrix}^T (\Omega + \lambda I)^{-1}T \quad (6)$$

where $K(X_i, X_j)$ is a Gaussian kernel function:

$$K(X_i, X_j) = \exp\left(-\frac{\|X_i - X_j\|^2}{\sigma^2}\right) \quad (7)$$

and σ is a kernel parameter. Compared to standard ELM, KELM does not need to specify the weight of the input layer or the number of neurons in the hidden layer, which reduces parameter tuning requirements and improves stability.

3 Algorithm for Solving Elliptic PDEs using KELM

3.1 Problem Formulation

Consider a standard elliptic PDE:

$$\frac{\partial^2 z}{\partial y^2} + \frac{\partial^2 z}{\partial x^2} = f(x, y, z), \quad a \leq x \leq b, \quad c \leq y \leq d \quad (8)$$

with Dirichlet boundary conditions:

$$z(a, y) = h_0(y), \quad z(b, y) = h_1(y), \quad z(x, c) = q_0(x), \quad z(x, d) = q_1(x) \quad (9)$$

3.2 Trial Solution Construction

The first key step in the KELM approach is to express the trial solution as a combination of boundary conditions and a KELM output:

$$z(x, y) = A(x, y) + B(x, y)u(x, y) \quad (10)$$

where:

$$\begin{aligned} A(x, y) = & \frac{b-x}{b-a}h_0(y) + \frac{x-a}{b-a}h_1(y) + \frac{d-y}{d-c} \left(q_0(x) - \left[\frac{b-x}{b-a}q_0(a) + \frac{x-a}{b-a}q_0(b) \right] \right) \\ & + \frac{y-c}{d-c} \left(q_1(x) - \left[\frac{b-x}{b-a}q_1(a) + \frac{x-a}{b-a}q_1(b) \right] \right) \end{aligned} \quad (11)$$

and

$$B(x, y) = (x-a)(b-x)(y-c)(d-y) \quad (12)$$

In this formulation, $u(x, y)$ is the output of the KELM. This construction ensures that the trial solution automatically satisfies the boundary conditions, regardless of the values of $u(x, y)$.

3.3 Error Function and Derivatives

To solve the PDE, an error function is defined:

$$\sum_{j=1}^N \left\| \frac{\partial^2 z_j}{\partial y^2} + \frac{\partial^2 z_j}{\partial x^2} - f(x_j, y_j, z_j) \right\| = 0 \quad (13)$$

The derivatives in this error function are calculated as:

$$\frac{\partial z}{\partial x} = \frac{\partial A}{\partial x} + \frac{\partial B}{\partial x}u + \frac{\partial u}{\partial x}B \quad (14)$$

$$\frac{\partial z}{\partial y} = \frac{\partial A}{\partial y} + \frac{\partial B}{\partial y}u + \frac{\partial u}{\partial y}B \quad (15)$$

$$\frac{\partial^2 z}{\partial x^2} = \frac{\partial^2 A}{\partial x^2} + \frac{\partial^2 B}{\partial x^2}u + 2\frac{\partial B}{\partial x}\frac{\partial u}{\partial x} + \frac{\partial^2 u}{\partial x^2}B \quad (16)$$

$$\frac{\partial^2 z}{\partial y^2} = \frac{\partial^2 A}{\partial y^2} + \frac{\partial^2 B}{\partial y^2}u + 2\frac{\partial B}{\partial y}\frac{\partial u}{\partial y} + \frac{\partial^2 u}{\partial y^2}B \quad (17)$$

3.4 KELM Formulation for PDEs

Applying the KELM approach and denoting $X = (x_j, y_j)_{j=1,2,\dots,N}$, the output u can be expressed as:

$$u_j = K(X_j, X)(\lambda I + \Omega)^{-1}T_j, \quad j = 1, 2, \dots, N \quad (18)$$

where:

$$T_j = - \left[\frac{\partial^2 A(X_j)}{\partial x^2} + \frac{\partial^2 A(X_j)}{\partial y^2} - f(X_j, z_j) \right] \quad (19)$$

and $\Omega = \{\Omega_{ij}\}_{i,j=1,2,\dots,N}$ with:

$$\begin{aligned} \Omega_{ij} = & K(X_i, X_j) + 2 \frac{\partial B(X_j)}{\partial x} \frac{\partial K(X_i, X_j)}{\partial x} + 2 \frac{\partial B(X_j)}{\partial y} \frac{\partial K(X_i, X_j)}{\partial y} \\ & + \frac{\partial^2 K(X_j)}{\partial x^2} B(x_j, y_j) + \frac{\partial^2 K(X_j)}{\partial y^2} B(x_j, y_j) \end{aligned} \quad (20)$$

The numerical solution to the elliptic PDE is then expressed as:

$$\hat{z}_j = A(X_j) + (x_j - a)(b - x_j)(y_j - c)(d - y_j)u_j \quad (21)$$

3.5 Algorithm Steps

The KELM algorithm for solving elliptic PDEs can be summarized as follows:

1. Select N training points in the rectangular area $[a, b] \times [c, d]$.
2. Construct trial solutions based on boundary conditions.
3. Solve for u_j using equation (17).
4. Calculate the numerical solution using equation (20).

The KELM contains two parameters that need to be specified: the regularization coefficient λ and kernel parameter σ . These parameters are determined using a grid search method, selecting values from the set $\{2^i | i = -9, -8, \dots, 10\}$ by tuning on a random 10% of the sample set.

The time complexity of the proposed method is $O(N^3)$, which primarily depends on solving the linear system of equations in step 3.

4 Numerical Examples and Results

The paper examines three numerical examples to demonstrate the performance of the KELM approach and compares it with existing methods. All examples consider domains of $[0, 1] \times [0, 1]$.

4.1 Example 1

The first example involves the PDE:

$$\frac{\partial^2 z}{\partial y^2} + \frac{\partial^2 z}{\partial x^2} = e^{-x}(x - 2 + y^3 + 6y), \quad 0 \leq x \leq 1, \quad 0 \leq y \leq 1 \quad (22)$$

with Dirichlet boundary conditions:

$$z(0, y) = y^3, \quad z(1, y) = (1 + y^3)e^{-1}, \quad z(x, 0) = xe^{-x}, \quad z(x, 1) = (x + 1)e^{-x} \quad (23)$$

The exact solution is $z = e^{-x}(x + y^3)$.

Using a 10×10 grid (121 training points), the KELM achieved absolute errors less than 10^{-6} . The mean absolute error (MAE), mean squared error (MSE), and standard deviation (SD) at the training points were approximately 1.02×10^{-7} , 2.85×10^{-14} , and 1.17×10^{-7} , respectively.

Compared to existing methods such as the Chebyshev neural network (ChNN), wavelet neural network optimized with particle swarm optimization (WNNPSO), wavelet neural network optimized with butterfly optimization algorithm (WNNBOA), and wavelet neural network optimized with improved butterfly optimization algorithm (WNNIBOA), the KELM demonstrated superior performance:

- KELM: MAE = 1.04×10^{-6} , MSE = 2.92×10^{-12} , SD = 1.35×10^{-6}
- WNNIBOA: MAE = 9.76×10^{-6} , MSE = 1.53×10^{-10} , SD = 7.57×10^{-6}
- WNNPSO: MAE = 1.93×10^{-5} , MSE = 6.48×10^{-10} , SD = 1.67×10^{-5}
- WNNBOA: MAE = 7.27×10^{-4} , MSE = 8.15×10^{-7} , SD = 5.36×10^{-4}
- ChNN: MAE = 1.26×10^{-2} , MSE = 2.72×10^{-4} , SD = 1.06×10^{-2}

4.2 Example 2

The second example considers the PDE:

$$\frac{\partial^2 z}{\partial y^2} + \frac{\partial^2 z}{\partial x^2} = -z(x^2 + y^2), \quad 0 \leq x \leq 1, \quad 0 \leq y \leq 1 \quad (24)$$

with Dirichlet boundary conditions:

$$z(0, y) = 0, \quad z(1, y) = \sin y, \quad z(x, 0) = 0, \quad z(x, 1) = \sin x \quad (25)$$

The exact solution is $z = \sin(xy)$.

Using the same grid as in Example 1, the KELM achieved absolute errors of less than 10^{-6} . The MAE, MSE, and SD values at the training points were approximately 5.23×10^{-7} , 1.14×10^{-13} , and 3.78×10^{-7} , respectively.

Comparative performance with other methods on untrained test points showed:

- KELM: MAE = 3.80×10^{-6} , MSE = 2.08×10^{-11} , SD = 2.52×10^{-6}
- WNNIBOA: MAE = 6.17×10^{-6} , MSE = 5.66×10^{-11} , SD = 4.30×10^{-6}
- WNNPSO: MAE = 3.07×10^{-5} , MSE = 1.35×10^{-9} , SD = 2.01×10^{-5}
- WNNBOA: MAE = 5.29×10^{-4} , MSE = 4.74×10^{-7} , SD = 4.41×10^{-4}

4.3 Example 3

The third example examines the PDE:

$$\frac{\partial^2 z}{\partial y^2} + \frac{\partial^2 z}{\partial x^2} = \sin(\pi y) \sin(\pi x), \quad 0 \leq x \leq 1, \quad 0 \leq y \leq 1 \quad (26)$$

with homogeneous Dirichlet boundary conditions:

$$z(0, y) = 0, \quad z(1, y) = 0, \quad z(x, 0) = 0, \quad z(x, 1) = 0 \quad (27)$$

The exact solution is $z = -\frac{1}{2\pi^2} \sin(\pi x) \sin(\pi y)$.

For this example, the KELM was compared with the ChNN method, and it demonstrated superior performance:

- KELM: MAE = 3.39×10^{-6} , MSE = 1.73×10^{-11} , SD = 2.42×10^{-6}
- ChNN: MAE = 2.00×10^{-3} , MSE = 6.32×10^{-6} , SD = 1.52×10^{-3}

4.4 Effect of Grid Size

The paper also investigated the effect of grid size (number of training points) on the performance of the KELM method. Four grid sizes were tested: 5×5 (36 training points), 10×10 (121 training points), 15×15 (225 training points), and 20×20 (441 training points).

The results showed that:

- As the grid was refined, the MAE, MSE, and SD of the proposed method decreased.
- The performance of the method with a 10×10 grid was very close to that with a 20×20 grid.
- The method requires only a few training points to achieve high computational accuracy, making it computationally efficient.

5 Application to Water Flow in Unsaturated Soils

As a practical application, the KELM method was applied to simulate water flow in unsaturated soils, a significant problem in engineering geology. The governing equation for steady-state water flow is:

$$\frac{\partial}{\partial x} \left(K_x(h) \frac{\partial h}{\partial x} \right) + \frac{\partial}{\partial y} \left(K_y(h) \frac{\partial h}{\partial y} + K_y(h) \right) = 0 \quad (28)$$

where $K(h) = K_s e^{\alpha_g h}$, with K_s being the saturated permeability coefficient of soils ($K_s = 1 \times 10^{-4}$ m/s) and α_g the pore-size distribution parameter ($\alpha_g = 8 \times 10^{-3}$).

The boundary conditions for this problem were:

$$h(0, y) = h_d, \quad h(W, y) = h_d, \quad h(x, 0) = h_d, \quad h(x, L) = \frac{1}{\alpha_g} \ln \left((1 - e^{\alpha_g h_d}) \sin \left(\frac{\pi x}{W} \right) + e^{\alpha_g h_d} \right) \quad (29)$$

where $h_d = -1$ m, $L = 1$ m, and $W = 1$ m.

The KELM method produced numerical solutions that were highly consistent with the analytical solution, demonstrating its ability to handle practical engineering problems effectively.

6 Implications and Advantages

The KELM approach offers several significant advantages for solving elliptic PDEs:

1. **Parameter Efficiency:** The KELM requires only two parameters (regularization coefficient λ and kernel parameter σ), compared to numerous parameters in neural network-based methods. This makes KELM more efficient and easier to tune.
2. **Computational Efficiency:** The KELM achieves a closed-form solution without requiring iterative optimization procedures, significantly reducing computational load compared to methods based on neural networks with gradient-based or heuristic learning algorithms.
3. **Accuracy:** Across all numerical examples, the KELM demonstrated superior accuracy compared to existing methods, including ChNN, WNNPSO, WNNBOA, and WNNIBOA, achieving error rates typically 1-4 orders of magnitude lower.
4. **Training Efficiency:** The KELM requires relatively few training points to achieve high computational accuracy, which further enhances its computational efficiency.

5. **Stability:** The incorporation of the kernel method and regularization improves the stability and generalization ability of the ELM approach, making it more robust for solving PDEs.
6. **Automatic Boundary Condition Satisfaction:** The trial solution construction automatically satisfies the boundary conditions, simplifying the overall solution process.

These advantages make the KELM a promising approach for solving elliptic PDEs in various engineering applications, particularly when computational efficiency and high accuracy are required.